

Software Test Plan (STP)

Smart Complaint Management System

Project: Smart Complaint Management System (Group 8)

Version: 1.0

Authors: Shasank SK (PES2UG24CS460),
Shreya Shenoy (PES2UG24CS487),
Sanjan M (PES2UG24CS442),
Samartha MS (PES2UG24CS435)

Date: 17-9-2026

Status: Draft

1. Introduction

Purpose:

This document defines the test plan for Version 1.0 of the Smart Complaint Management System (Group 8). It outlines the testing objectives, scope, strategy, environment, schedule, deliverables, and responsibilities needed to verify that the system meets the functional requirements (REQ-1.1 through REQ-13.6) and the non-functional requirements (performance, safety, security, quality attributes, business rules, and domain requirements) defined in the Group 8 SRS v1.0.

Scope:

Testing covers all 13 functional features defined in the SRS: user registration and login, complaint registration, complaint categorization, complaint tracking and notifications, complaint assignment, complaint escalation, smart complaint classification (priority), duplicate complaint detection, complaint status updates, admin dashboard, reports and analytics, role-based access control, and search and filtering. Testing is scoped to the standalone academic prototype (Python/Flask/MySQL, browser-based client). External government/municipal integration, hardware interfaces, and emergency/crime-handling workflows are explicitly excluded, consistent with the scope boundaries stated in the Project Proposal and SRS.

References:

- Group 8 Software Requirements Specification (SRS_g8.pdf), Version 1.0 approved
- Group 8 Software Engineering Project Proposal, Version 1.0
- Use Case Diagram and Activity Diagram, Team 8, Version 1.0

Definitions:

- RBAC — Role-Based Access Control
- SRS — Software Requirements Specification
- RTM — Requirements Traceability Matrix
- UAT — User Acceptance Testing
- NFR — Non-Functional Requirement

2. Test Items

- User Registration and Login module
- Complaint Registration module
- Complaint Categorization module
- Complaint Tracking and Notification module
- Complaint Assignment module

- Complaint Escalation module
- Smart Complaint Classification (priority) module
- Duplicate Complaint Detection module
- Complaint Status Update module
- Admin Dashboard module
- Reports and Analytics module
- Role-Based Access Control (RBAC) module
- Search and Filtering module

3. Features to be Tested

Features mapped to SRS requirement IDs:

Feature	Requirement IDs
5.1 User Registration and Login	REQ-1.1 – REQ-1.7
5.2 Complaint Registration	REQ-2.1 – REQ-2.6
5.3 Complaint Categorization	REQ-3.1 – REQ-3.5
5.4 Complaint Tracking and Notifications	REQ-4.1 – REQ-4.5
5.5 Complaint Assignment	REQ-5.1 – REQ-5.5
5.6 Complaint Escalation	REQ-6.1 – REQ-6.5
5.7 Smart Complaint Classification (Priority)	REQ-7.1 – REQ-7.5
5.8 Duplicate Complaint Detection	REQ-8.1 – REQ-8.5
5.9 Complaint Status Updates	REQ-9.1 – REQ-9.6
5.10 Admin Dashboard	REQ-10.1 – REQ-10.6
5.11 Reports and Analytics	REQ-11.1 – REQ-11.5
5.12 Role-Based Access Control	REQ-12.1 – REQ-12.6
5.13 Search and Filtering	REQ-13.1 – REQ-13.6
NFR-P1 – NFR-P5 (Performance)	Section 6.1 of SRS
NFR-SEC1 – NFR-SEC8 (Security)	Section 6.3 of SRS

4. Features Not to be Tested

- External government/municipal complaint-system integration — explicitly out of scope per the Project Proposal and SRS; no such integration exists in this prototype
- Emergency/crime-handling workflows — the SRS explicitly states these are outside system scope and must be directed to emergency/law-enforcement services
- Hardware interfaces — the SRS explicitly states no hardware interface is implemented for this project
- Internal reliability of third-party notification delivery infrastructure (e.g., underlying email/SMS gateway uptime) — only the system's correct triggering of notifications is tested, not the gateway provider's infrastructure
- Browser rendering engine internals — Chrome/Firefox/Edge are assumed to render standard HTML/CSS correctly; only application-level compatibility is tested
- Large-scale/nationwide load and stress testing — the SRS scopes this system as an academic proof-of-concept, not a production-scale deployment
- Multilingual support, voice-based filing, chatbot assistance — listed as future enhancements outside current project scope

5. Test Approach / Strategy

Levels

- Unit tests — individual Flask route handlers and logic functions (e.g., priority-scoring function, duplicate-comparison function) tested in isolation
- Integration tests — Flask backend ↔ MySQL database interactions, and cross-feature pipelines (e.g., registration → categorization → priority classification → duplicate check)
- System tests — end-to-end complaint lifecycle: login → lodge complaint → categorize/classify → assign → status update → escalate (if needed) → resolve → notify → close
- Acceptance tests — informal UAT performed by the team and course evaluators against the functional feature list in the Project Proposal

Types

- Functional testing — verify each REQ-x.x behaves as specified, including invalid-input and error-message handling
- Regression testing — re-run core flows (auth, complaint registration, status updates) after each new feature is merged, given four members integrating in parallel
- Performance testing — verify response times against NFR-P1–NFR-P5 (registration, tracking, dashboard load, search/filter, status updates) under expected prototype-scale data volumes
- Usability testing — verify UI clarity, consistent navigation, and accessibility per the SRS's User Interface and Software Quality Attributes sections
- Compatibility testing — verify functionality on Chrome, Firefox, and Edge (latest versions), per the Operating Environment section of the SRS
- Accuracy testing — evaluate smart complaint classification (priority) and duplicate complaint detection results against a hand-labelled test set of sample complaints
- Security/access-control testing — see Section 5.1 below

Entry Criteria:

A stable build with authentication, RBAC, and the core complaint-registration pipeline integrated; test data (dummy user and admin accounts, sample complaints across all categories) prepared; Flask + MySQL test environment set up and reachable by all team members.

Exit Criteria:

100% of planned test cases executed; all High-priority feature defects (Features 5.1, 5.2, 5.4, 5.5, 5.9, 5.10, 5.12 per SRS priority ratings) resolved; RBAC correctly denies unauthorized access in all tested cases; all functional and non-functional requirements in the SRS verified; no critical/blocker defects open at the time of the final demonstration.

5.1 Security & Access-Control Validation

- Verify passwords are hashed and never stored or logged in plaintext (NFR-SEC2)
- Verify HTTPS is enforced wherever the deployment environment supports it (NFR-SEC3)
- Verify RBAC is enforced at the backend/API level — attempt direct URL/endpoint access as a Citizen to admin-only routes and confirm denial (NFR-SEC4, REQ-12.4–REQ-12.6)
- Verify session handling: session expiry on inactivity and invalidation on logout (NFR-SEC1, NFR-SEC6)
- Verify input validation/sanitization on complaint and registration forms rejects malformed or malicious input without exposing stack traces or internal errors
- Verify unauthenticated requests to protected functions are denied with an appropriate error rather than partial data exposure (REQ-1.6, NFR-SEC6)
- Verify personal/complaint data is not exposed beyond what each role is authorized to see (NFR-SEC5, BR9)

6. Test Environment

Hardware:

Standard desktop/laptop computers used for development and testing. No specialized hardware is required — the SRS explicitly states no hardware interface is implemented for this project.

Software:

- Python runtime and Flask application (backend under test)
- MySQL database server (dedicated test instance, separate from any development data)
- Modern web browsers for compatibility testing: Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions)

Tools:

- pytest (or unittest) — backend unit and integration tests
- Flask test client — route-level testing without a running server
- Postman — manual/automated API endpoint testing
- GitHub Issues — defect tracking, linked to the team's existing repository

Test Data:

- Dummy user accounts (multiple Citizens, at least one Admin)
- Sample complaints covering every predefined category, including edge cases (missing fields, oversized text, duplicate/near-duplicate descriptions, boundary priority-triggering keywords)

7. Test Schedule

Aligned with the four-week Plan of Work in the Project Proposal. Replace the placeholder dates below with your actual calendar dates once the project start date is fixed.

Period	Focus	Planned Testing Work
Week 1	Foundation & data model	Unit-test auth/registration/login as they're built; agree on test data conventions and shared environment setup
Week 2	Core complaint workflow	Functional + integration testing of complaint registration, categorization, tracking, notifications, statuses; begin regression suite
Week 3	Admin workflow & smart features	Functional testing of assignment, escalation, admin dashboard, search/filtering, reports; accuracy testing of priority classification and duplicate detection; security/RBAC validation
Week 4	Integration, system testing & UAT	Full end-to-end system tests across the complete complaint lifecycle; performance and compatibility testing; defect fixing; final Test Summary Report; UAT/demo rehearsal

8. Test Deliverables

- Test Plan (this document)
- Test cases mapped to REQ-x.x IDs (manual and/or automated)
- Test scripts (pytest/Selenium)
- Test data sets
- Test execution logs
- Defect reports (GitHub Issues)
- Updated Requirements Traceability Matrix (extends SRS Appendix C)
- Final Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Shasank SK	Coordinates the test plan, tracks overall execution status, and owns security/RBAC test validation (consistent with his Security & Privacy ownership in the Proposal)
Test Engineer — Auth & Access	Shasank SK	Designs and executes test cases for user registration/login, RBAC, complaint registration, and complaint categorization
Test Engineer — Lifecycle & Notifications	Shreya Shenoy	Designs and executes test cases for complaint tracking/notifications, escalation, and status updates
Test Engineer — Smart Features & Search	Sanjan M	Designs and executes test cases for priority classification, duplicate detection, and search/filtering, including accuracy evaluation
Test Engineer — Admin & Data	Samartha MS	Designs and executes test cases for complaint assignment, admin dashboard, and reports/analytics

10. Risks and Mitigation

Risk	Mitigation
Smart features (priority classification, duplicate detection) not built until Week 3, compressing time for accuracy testing	Prepare a labelled sample-complaint test set during Week 1–2 so accuracy testing can start the moment the feature is merged
Four members integrating independently built modules could cause interface mismatches	Enforce the shared interfaces/conventions agreed in Week 1; run integration tests immediately after each merge, not only at the end
Inconsistent local dev environments (Python/Flask/MySQL versions) across members	Maintain a shared requirements file and environment setup doc; verify environment parity before each test cycle
RBAC or auth defects discovered late could block testing of every other feature	Prioritize auth/RBAC testing in Week 1–2 since almost every other feature depends on a logged-in, role-identified user
Limited time to cover all 13 features and their NFRs by the final demo	Test High-priority features (per SRS priority ratings) first; treat Low-priority features (Reports & Analytics, Search & Filtering) as acceptable to finish testing last

11. Assumptions & Dependencies

Assumptions:

- Testers (team members) have access to a computer with a supported modern browser and a stable internet connection
- Test data (dummy accounts, sample complaints) will be prepared before each test cycle begins
- Admin actions (assignment, status updates, escalation review) will continue to be performed manually during testing, consistent with the SRS assumption that administrators log in manually
- Predefined complaint categories used in the SRS cover the test scenarios needed; edge cases will be tested under the "Other" category

Dependencies:

- Availability and stability of the Flask application and Python runtime environment
- Availability and stability of the MySQL test database
- Availability of the configured notification mechanism (or a test/mock substitute) for verifying notification-triggering logic
- A stable network connection between the browser client, Flask backend, and MySQL server during test execution

12. Suspension & Resumption Criteria

Suspend testing if:

- The shared test environment (Flask/MySQL) is unavailable for more than 4 hours
- The build is unstable enough to block more than 30% of planned test cases
- A critical defect is found in authentication or RBAC that would invalidate results for every other feature

Resume testing if:

- The blocking defect(s) are resolved and verified
- The test environment is restored and confirmed stable

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM) extends Appendix C of the SRS, mapping each REQ-x.x to one or more test case IDs. Example mappings across features:

Requirement ID(s)	Feature	Test Case ID(s)
REQ-1.1 – REQ-1.7	User Registration and Login	TC-Auth-01 ... TC-Auth-07
REQ-2.1 – REQ-2.6	Complaint Registration	TC-Reg-01 ... TC-Reg-06
REQ-7.1 – REQ-7.5	Smart Complaint Classification	TC-Prio-01 ... TC-Prio-05
REQ-8.1 – REQ-8.5	Duplicate Complaint Detection	TC-Dup-01 ... TC-Dup-05
REQ-12.1 – REQ-12.6	Role-Based Access Control	TC-RBAC-01 ... TC-RBAC-06
NFR-P1 – NFR-P5	Performance Requirements	TC-Perf-01 ... TC-Perf-05

14. Test Metrics & Reporting

Metrics collected:

- % of test cases executed
- % passed / failed
- Defect density (defects per feature)
- Defect aging (time open)
- Requirement coverage (REQ-x.x with at least one passing test case)

Reports:

- Weekly execution status update (aligned with the Project Proposal's Week 1–4 plan)
- Final Test Summary Report, delivered before the final demonstration

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		